

UNIVERSITY OF CLAUDE · FREE RESOURCE MAP

Resource Map

CSCE 713 Software Security — module-by-module mapping onto free university courseware

HOW TO USE THIS

This is your lecture hall. For each module: watch or read the mapped external material first, then work the module slides and notes, which are written to assume you have seen it.

Where a specific lecture number is given, that is the one that covers this module. Where a book chapter is given, read that chapter, not the whole book.

Everything here is free. Resources remain under their own licences — link to them, do not redistribute them.

Primary sources for the whole course

Resource	Role in this course
Dowd, McDonald & Schuh — The Art of Software Security Assessment	The primary reference for Modules 02 through 06 and 11. Dated in its examples and unmatched in its treatment of how defects arise and how to find them by reading. Library copy.
Zeller et al. — The Fuzzing Book (free, interactive)	Module 08's primary source, free and executable. Builds fuzzers from scratch in the browser, which is the right way to understand what coverage guidance actually does.
Møller & Schwartzbach — Static Program Analysis (free PDF)	Module 07's theory, free. The lattice and fixpoint material from CSCE 605 applied to defect detection, and honest about the precision limits.
Google — Building Secure and Reliable Systems, the development chapters (free PDF)	Modules 10 and 12, free in full — and notable for arguing that secure defaults in frameworks beat developer education, which is this course's position too.
The CWE and the OWASP Top Ten (free)	The shared vocabulary for Modules 02 through 06. Use CWE identifiers when reporting; it makes a finding searchable and comparable.
Erickson — Hacking: The Art of Exploitation, 2nd edition	Optional, for Module 12's mitigations. Understanding why a mitigation works requires understanding what it stops — and the exercises are for your own machine and the book's own targets only.

Module-by-module map

Module 01 — Why Software Has Vulnerabilities

Resource	What it covers for this module
Dowd, McDonald & Schuh, chapters 1 and 2	§1 and §2's method — the best treatment of reading code adversarially that exists. Library copy.
The CWE Top 25 and its methodology (free)	§3's severity factors and the evidence for Modules 02 through 06's ordering — including how the ranking is computed, which is worth understanding.
CVSS specification (free)	§3 as the industry's scoring system — useful, widely required, and read CSCE 701 Module 12 on why a single number misleads.
MIT 6.858, the first lectures (free, OCW)	§1 and §2 as lectures , free, with the threat model framing developed carefully.

Module 02 — Memory Safety

Resource	What it covers for this module
Dowd, McDonald & Schuh, chapters 5 through 7	§2 and §3 in depth — the mechanisms explained at the level of what the machine does. Library copy.
Szekeres et al. — SoK: Eternal War in Memory (free)	§1's taxonomy and §3's mitigation history , systematised — the single best overview of this material.
MIT 6.858's buffer overflow lab (free, OCW)	§2 and §3 practically , on targets built for the purpose — which is Module 01 §4's constraint satisfied.
Google and Microsoft's memory safety reports (free)	§4's evidence — measured proportions of severe defects attributable to memory safety, from codebases at scale.

Module 03 — Integers and Types

Resource	What it covers for this module
Dowd, McDonald & Schuh, chapter 6	§1 and §2 exhaustively — the best treatment of C's integer behaviour in a security context. Library copy.
Wang et al. — Undefined Behavior: What Happened to My Code? (free)	§2's last point, demonstrated — real security checks deleted by real compilers, with the mechanism.
CERT C Coding Standard, the INT rules (free)	§2's practice as a rule set , with compliant and non-compliant examples for each case.
CWE-190, CWE-680, CWE-843 (free)	§1 and §4 with the shared vocabulary — use these identifiers when reporting.

Module 04 — Injection and Parsing

Resource	What it covers for this module
OWASP — the injection cheat sheets (free)	§1 through §3 as current, specific guidance per destination — maintained, and the right reference to hand a team.
King — Parse, Don't Validate (free)	§4 in the original — short, and it reframes input handling permanently.
Dowd, McDonald & Schuh, chapters 8 and 17	§2 and §3's specifics — encoding confusion and path handling, in more detail than anywhere else. Library copy.
Google's safe-by-construction libraries writeup (free)	§4 at scale — types that make injection unrepresentable, deployed across a large codebase.

Module 05 — Concurrency Defects

Resource	What it covers for this module
Boehm — Threads Cannot Be Implemented As a Library (free)	§1's argument for why the memory model must be in the language — and why a data race is undefined rather than merely unlucky.
Lu et al. — Learning from Mistakes: a study of real-world concurrency bugs (free)	§2's classification, measured — the study that established atomicity violations as the dominant real-world class.
ThreadSanitizer documentation (free)	§4's primary tool — what it detects, what it misses, and its cost.
CWE-362, CWE-367, CWE-416 (free)	§2 and §3 with the shared vocabulary — race condition, TOCTTOU, and use-after-free.

Module 06 — Deserialisation and Trust Boundaries

Resource	What it covers for this module
Frohoff & Lawrence — Marshalling Pickles (free)	§1's gadget chains in the original — the talk that made this class widely understood.
OWASP — Deserialization and SSRF cheat sheets (free)	§2 and §3 as current, per-platform guidance , maintained and specific.
Google — the Java serialization deprecation rationale (free)	§1's severity, as judged by the platform's own maintainers — a language feature deprecated for security reasons, which is rare.
CWE-502, CWE-918 (free)	§1 and §3 with the shared vocabulary.

Module 07 — Static Analysis

Resource	What it covers for this module
Møller & Schwartzbach — Static Program Analysis (free PDF)	§1 and §2's theory — free, rigorous, and explicit about the precision limits.

Resource	What it covers for this module
CodeQL documentation and the query examples (free)	§4's practice — and the query language is worth learning for the project-specific rules alone.
Semgrep registry and rule-writing guide (free)	§4 with a much shallower learning curve — the right starting point for a first custom rule.
Bessey et al. — A Few Billion Lines of Code Later (free)	§3's deployment reality , from the people who commercialised it — honest about why adoption fails, and the best thing written on this.

Module 08 — Fuzzing

Resource	What it covers for this module
The Fuzzing Book (free, interactive)	The whole module, executable — build a coverage-guided fuzzer yourself, which is the fastest route to understanding §1.
libFuzzer and AFL++ documentation (free)	§2 and §3 in practice — the harness rules and the coverage tooling, from the implementations.
OSS-Fuzz, and its ideal integration guide (free)	§4's continuous operation , plus the best available guidance on harness design at scale.
Klees et al. — Evaluating Fuzz Testing (free)	§3's argument, rigorously — why duration and crash counts are poor measures, and what to report instead.